Control cuántico

Genesis Silesky

21 de octubre de 2026

1. Control cuántico: Hamiltoniano, rotaciones y acoplamiento

1.1. Objetivo

Entender de dónde salen las puertas cuánticas: cada una es una evolución unitaria $U(t) = e^{-iHt}$ que se consigue eligiendo el Hamiltoniano H. Se ve cómo se hacen rotaciones de un qubit, qué pasa con un pulso desafinado y cómo el acoplamiento entre dos qubits los entrelaza.

1.2. Conceptos previos

Concepto previo: unitariedad, matrices de Pauli y producto tensorial

Ver sección **??** (matrices unitarias y producto tensorial) y los capítulos 10 y 11. Las matrices de Pauli son $\sigma_x = \left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right)$, $\sigma_y = \left(\begin{smallmatrix} 0 & -i \\ i & 0 \end{smallmatrix}\right)$ y $\sigma_z = \left(\begin{smallmatrix} 1 & 0 \\ 0 & -1 \end{smallmatrix}\right)$.

1.3. Fundamento teórico

Unitariedad. Una operación U es unitaria si $U^\dagger U = I$. Los postulados de la mecánica cuántica dicen que toda evolución física, incluidas las operaciones lógicas de un computador cuántico, es unitaria. Su significado físico es que la suma de probabilidades se conserva:

$$\langle U\psi|U\psi\rangle = \langle\psi|U^\dagger U|\psi\rangle = \langle\psi|\psi\rangle = |\alpha|^2 + |\beta|^2. \tag{1}$$

No se pierde ni se crea nada: la operación no destruye información.

Hamiltoniano y evolución. El Hamiltoniano H es el operador de la energía total del sistema y manda su evolución en el tiempo (ecuación de Schrödinger):

$$i\hbar\frac{d}{dt}\,|\psi(t)\rangle = H\,|\psi(t)\rangle\,. \tag{2}$$

Con $\hbar = 1$ y un H constante, la solución es $|\psi(t)\rangle = U(t)\,|\psi(0)\rangle$ con

$$U(t) = e^{-iHt}. \tag{3}$$

Controlar un computador cuántico es, básicamente, modificar H para obtener la evolución $U(t)$ deseada. En general $H = H_0 + H_1$: la parte *interna* H_0 (energía Zeeman de cada qubit y acoplamiento entre ellos) y la parte *externa* H_1 (el campo de radiofrecuencia aplicado).

Un qubit: rotaciones. Un pulso resonante de duración t, frecuencia de Rabi ω_1 y fase φ produce

$$U = \exp\Big[-i\,\pi\omega_1 t\,\big(\cos\varphi\,\sigma_x + \sin\varphi\,\sigma_y\big)\Big], \tag{4}$$

una **rotación** alrededor del eje $\cos\varphi\,\hat{x} + \sin\varphi\,\hat{y}$ de la esfera de Bloch, por un ángulo $\theta = 2\pi\omega_1 t$. La fase del pulso elige el eje y la duración elige el ángulo. Se escribe $R_x(\theta)$, $R_y(\theta)$, etc.

La rotación alrededor de z. Un solo pulso no puede girar alrededor de z, pero se puede armar con tres:

1. girar $\pi/2$ alrededor de y (el eje x ocupa el lugar de z);
2. girar θ alrededor de x (equivale a girar alrededor de z);
3. girar $\pi/2$ alrededor de $-y$ para volver al sistema original.

Así, con pulsos de radiofrecuencia se puede girar un qubit alrededor de cualquier eje. El **orden de los pulsos importa**, porque las rotaciones, en general, no conmutan.

Pulso desafinado. Si la frecuencia del pulso difiere de la del qubit en $\Delta\omega$, la evolución combina dos giros a la vez: alrededor de z (a frecuencia $\Delta\omega$) y alrededor del eje del pulso. Con una desintonía grande el pulso no logra excitar al qubit.

Dos qubits acoplados. Con acoplamiento J_{12}, la parte interna del Hamiltoniano (en el sistema doblemente rotante) es $H_0 = 2\pi J_{12}\,\sigma_z \otimes \sigma_z/4$. Es una rotación alrededor del eje $\sigma_z \otimes \sigma_z$, que equivale a decir que *el eje de giro de un qubit depende del estado del otro*: si el otro está en $|0\rangle$ el eje es $+z$ y si está en $|1\rangle$ es $-z$. Si el otro qubit está en una superposición, la frecuencia de precesión también queda en superposición y los dos qubits se **entrelazan**.

Criterio de entrelazamiento. Si los qubits no están entrelazados, el estado total es un producto $\rho = \rho_A \otimes \rho_B$. Si ρ no se puede escribir así, el sistema está entrelazado.

1.4. Código (completamente comentado)

```
# %% [markdown]
# # 12 - Control cuantico: Hamiltoniano, rotaciones y acoplamiento entre qubits

# %% Celda 1: importaciones
import numpy as np
from scipy.linalg import expm
# Statevector es el estado; partial_trace y entropy sirven para ver si hay entrelazamiento.
from qiskit.quantum_info import DensityMatrix, Statevector, entropy, partial_trace

# Matrices de Pauli. Trabajamos con hbar = 1 y frecuencias en Hz (por eso aparece 2*pi).
sx = np.array([[0, 1], [1, 0]], dtype=complex)
sy = np.array([[0, -1j], [1j, 0]], dtype=complex)
sz = np.array([[1, 0], [0, -1]], dtype=complex)

# %% Celda 2: la ecuacion de Schrodinger da una operacion unitaria
# El Hamiltoniano H es la energia total del sistema y manda su evolucion:
#     i d|psi>/dt = H |psi>.
# Si H no cambia en el tiempo, la solucion es  |psi(t)> = U(t)|psi(0)>  con
#     U(t) = exp(-i H t).
# "Controlar" un qubit es cambiar H para obtener la U que queremos.
H = 2 * np.pi * 1000 * (0.8 * sx + 0.6 * sz) / 2   # un H cualquiera
U = expm(-1j * H * 0.3e-3)
print("U^dagger U = I (es unitaria):", np.allclose(U.conj().T @ U, np.eye(2)))

# Unitaria significa que la suma de probabilidades sigue valiendo 1: no se
# pierde ni se crea nada, y por eso la operacion se puede invertir.
psi = np.array([0.6, 0.8j])
print("Suma de probabilidades antes  :", round(float(np.sum(abs(psi) ** 2)), 6))
print("Suma de probabilidades despues:", round(float(np.sum(abs(U @ psi) ** 2)), 6))


# %% Celda 3: rotaciones de un qubit con pulsos
```

```python
# Un pulso de duracion t, con frecuencia de Rabi w1 y fase phi, aplica
#     U = exp(-i * 2*pi*w1 * (cos(phi) sx + sin(phi) sy)/2 * t)
# o sea, una ROTACION de angulo  theta = 2*pi*w1*t  alrededor del eje
# cos(phi) x + sin(phi) y.  phi = 0 -> eje x;  phi = pi/2 -> eje y.
def rotacion(theta, phi):
    """Rotacion de angulo theta alrededor del eje cos(phi) x + sin(phi) y."""
    return expm(-1j * theta / 2 * (np.cos(phi) * sx + np.sin(phi) * sy))


def Rx(theta):
    return rotacion(theta, 0.0)


def Ry(theta):
    return rotacion(theta, np.pi / 2)


def Rz(theta):
    return expm(-1j * theta / 2 * sz)   # un pulso solo no la da directamente (ver Celda 4)


w1, t = 1000.0, 0.25e-3                      # pulso de 0.25 ms a 1000 Hz
theta = 2 * np.pi * w1 * t
print(f"theta = 2*pi*w1*t = {theta:.4f} rad (pi/2 = {np.pi / 2:.4f})")
print("El pulso coincide con Rx(pi/2):", np.allclose(expm(-1j * 2 * np.pi * w1 * sx / 2 * t), Rx
    ↪ (np.pi / 2)))

# %% Celda 4: la rotacion alrededor de z se arma combinando tres pulsos
# Un pulso solo puede girar alrededor de ejes del plano xy, no de z. Truco:
#   1. Girar pi/2 alrededor de y   (el eje x pasa a ocupar el lugar de z).
#   2. Girar theta alrededor de x  (equivale a girar alrededor de z).
#   3. Girar pi/2 alrededor de -y  (volver al sistema original).
# El orden importa, porque las rotaciones no conmutan.
theta_z = 0.9
# En un producto de matrices, el primer pulso aplicado queda a la DERECHA.
secuencia = Ry(-np.pi / 2) @ Rx(theta_z) @ Ry(np.pi / 2)
cambiada = Ry(np.pi / 2) @ Rx(theta_z) @ Ry(-np.pi / 2)
print("Ry(pi/2), Rx(th), Ry(-pi/2) == Rz(+th):", np.allclose(secuencia, Rz(theta_z)))
print("Ry(pi/2), Rx(th), Ry(-pi/2) == Rz(-th):", np.allclose(secuencia, Rz(-theta_z)))
print("Con el orden cambiado        == Rz(+th):", np.allclose(cambiada, Rz(theta_z)),
      "| == Rz(-th):", np.allclose(cambiada, Rz(-theta_z)))

# %% Celda 5: un pulso desafinado (fuera de resonancia)
# Si la frecuencia del pulso difiere de la del qubit en dw, aparece un giro
# extra alrededor de z:   H = 2*pi*(dw sz + w1 sx)/2.
# Con desintonia grande el pulso casi no logra voltear al qubit.
ket0 = np.array([1, 0], dtype=complex)
for dw in (0.0, 1000.0, 10000.0):
    Hd = 2 * np.pi * (dw * sz + w1 * sx) / 2
    p1 = abs((expm(-1j * Hd * 0.5e-3) @ ket0)[1]) ** 2   # pulso "pi" de 0.5 ms
    print(f"desintonia = {dw:7.0f} Hz -> P(1) tras un pulso pi = {p1:.3f}")

# %% Celda 6: dos qubits acoplados
# Cuando dos qubits interactuan con acoplamiento J, el Hamiltoniano es
#     H = 2*pi*J * (sz (x) sz)/4.
# Es una rotacion alrededor del eje sz(x)sz: el eje de giro de un qubit
# DEPENDE del estado del otro (+z si el otro es |0>, -z si es |1>).
J = 696.0                          # acoplamiento en Hz
zz = np.kron(sz, sz)


def U_acople(t):
    return expm(-1j * 2 * np.pi * J * zz / 4 * t)
```

```
95
96
97  # El qubit 0 empieza en |+> (sobre el ecuador). Segun el estado del otro qubit,
98  # acumula una fase relativa de signo contrario.
99  t = 1 / (2 * J)                 # con este tiempo el angulo de rotacion es pi/2
100 mas = np.array([1, 1], dtype=complex) / np.sqrt(2)
101 for otro, nombre in ((np.array([1, 0], dtype=complex), "|0>"), (np.array([0, 1], dtype=complex),
        ↪ "|1>")):
102     # np.kron(otro, mas): el factor de la izquierda es el qubit 1 y el de la derecha el qubit 0.
103     estado = Statevector(U_acople(t) @ np.kron(otro, mas))
104     rho = partial_trace(DensityMatrix(estado), [1]).data   # qubit 0 solo
105     x, y = np.real(np.trace(rho @ sx)), np.real(np.trace(rho @ sy))
106     print(f"otro qubit en {nombre}: qubit 0 queda en (x, y) = ({x:+.2f}, {y:+.2f})")
107
108 # %% Celda 7: el acoplamiento entrelaza
109 # Desde |++>, tras t = 1/(2J) los dos qubits quedan entrelazados. Criterio:
110 # un estado NO entrelazado se puede escribir como producto de los estados de
111 # cada qubit (rho = rho_A (x) rho_B). Si no se puede, hay entrelazamiento.
112 inicial = np.kron(mas, mas)
113 for t in (0.0, 1 / (2 * J)):
114     rho = DensityMatrix(Statevector(U_acople(t) @ inicial))
115     rho_A, rho_B = partial_trace(rho, [1]), partial_trace(rho, [0])
116     es_producto = np.allclose(rho.data, np.kron(rho_B.data, rho_A.data))
117     print(f"t = {t * 1e3:.3f} ms: rho == rho_A (x) rho_B? {es_producto} | "
118           f"entropia del qubit 0 = {float(entropy(rho_A, base=2)):.3f} bit")
```

1.5. Explicación celda por celda

Celda 2 Calcula $U = e^{-iHt}$ para un H cualquiera y comprueba $U^\dagger U = I$ y que la suma de probabilidades sigue valiendo 1.

Celda 3 Define R_x, R_y, R_z y verifica que un pulso de 0.25 ms a 1000 Hz es $R_x(\pi/2)$.

Celda 4 Arma $R_z(\theta)$ con la secuencia de tres pulsos y comprueba que el orden contrario da $R_z(-\theta)$.

Celda 5 Aplica un pulso π con distintas desintonías.

Celda 6 Con $J = 696$ Hz y $t = 1/(2J)$ deja al qubit 0 en $|+\rangle$ y muestra que su fase cambia de signo según el estado del qubit 1.

Celda 7 Comprueba, desde $|{+}{+}\rangle$, que tras $t = 1/(2J)$ el estado ya no es un producto.

1.6. Salida obtenida

```
U^dagger U = I (es unitaria): True
Suma de probabilidades antes  : 1.0
Suma de probabilidades despues: 1.0
theta = 2*pi*w1*t = 1.5708 rad (pi/2 = 1.5708)
El pulso coincide con Rx(pi/2): True
Ry(pi/2), Rx(th), Ry(-pi/2) == Rz(+th): True
Ry(pi/2), Rx(th), Ry(-pi/2) == Rz(-th): False
Con el orden cambiado        == Rz(+th): False | == Rz(-th): True
desintonia =       0 Hz -> P(1) tras un pulso pi = 1.000
desintonia =    1000 Hz -> P(1) tras un pulso pi = 0.317
desintonia =   10000 Hz -> P(1) tras un pulso pi = 0.000
otro qubit en |0>: qubit 0 queda en (x, y) = (+0.00, +1.00)
otro qubit en |1>: qubit 0 queda en (x, y) = (+0.00, -1.00)
t = 0.000 ms: rho == rho_A (x) rho_B? True | entropia del qubit 0 = 0.000 bit
t = 0.718 ms: rho == rho_A (x) rho_B? False | entropia del qubit 0 = 1.000 bit
```

Es determinista (no hay medición).

1.7. Interpretación

La suma de probabilidades queda en 1 antes y después de U. Los tres pulsos $R_y(\pi/2)$, $R_x(\theta)$, $R_y(-\pi/2)$ dan $R_z(+\theta)$, y con el orden cambiado se obtiene $R_z(-\theta)$. Con desintonía de 1000 Hz un pulso π solo logra $P(1) = 0.317$, y con 10 000 Hz, 0.000. Con el qubit 1 en $|0\rangle$ o en $|1\rangle$, el qubit 0 acaba en $(x, y) = (0, +1)$ o $(0, -1)$: fases opuestas. Tras $t = 0.718$ ms el estado ya no es un producto y la entropía de un qubit es 1 bit (máximo).

1.8. Ejercicios

1. Construye $R_z(\theta)$ con el orden contrario de pulsos ($R_y(-\pi/2)$ primero) y comprueba qué signo obtienes.
2. ¿Qué tiempo t hace falta con $J = 696$ Hz para una rotación de π en el eje $\sigma_z \otimes \sigma_z$?
3. En la Celda 7, calcula la entropía para varios tiempos entre 0 y $1/J$ y observa cuándo es máxima.

1.9. Errores comunes

- Multiplicar las matrices en el orden en que se aplican los pulsos: en un producto de matrices el primer pulso aplicado queda a la *derecha*.
- Con `np.kron(A, B)`, el factor de la izquierda es el qubit 1 y el de la derecha el qubit 0 (orden de Qiskit).